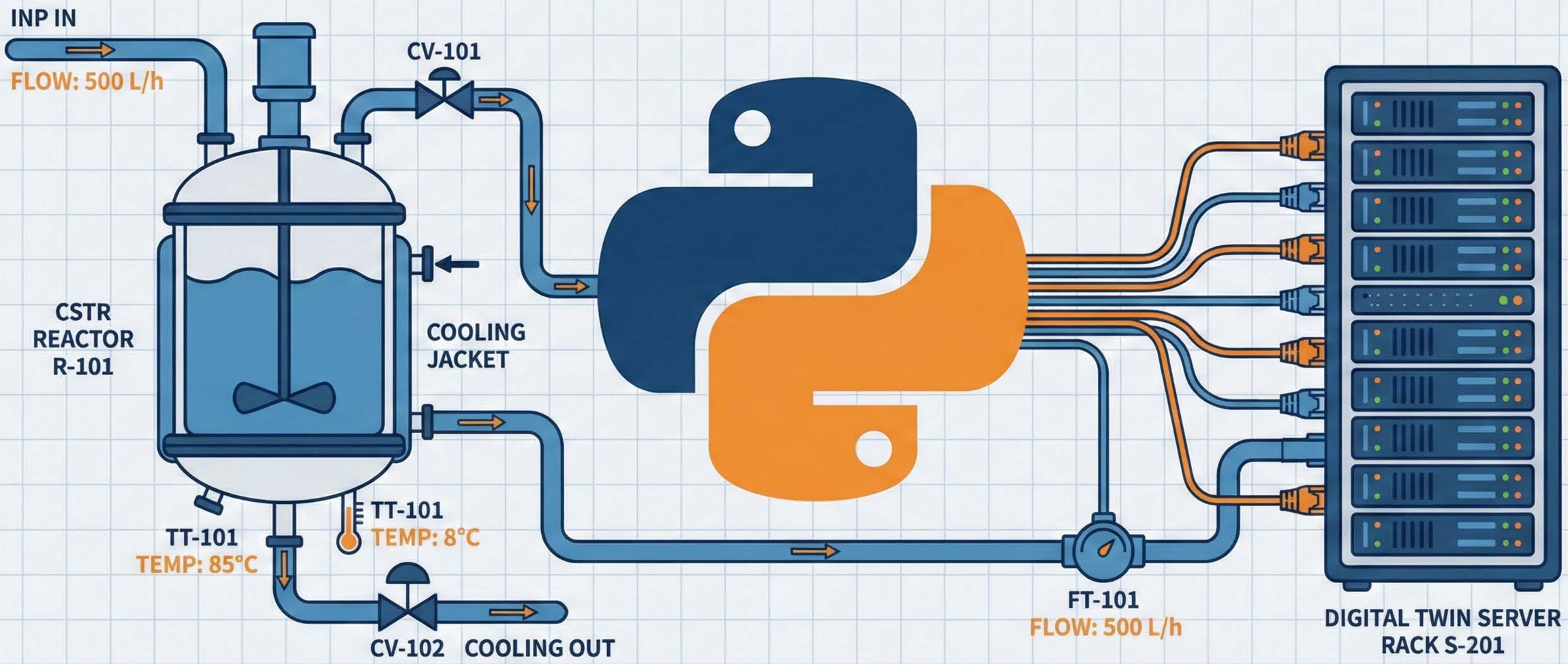
Python 程式語言基礎：建構數位化工廠的基石

Unit 02 語法核心與資料科學應用



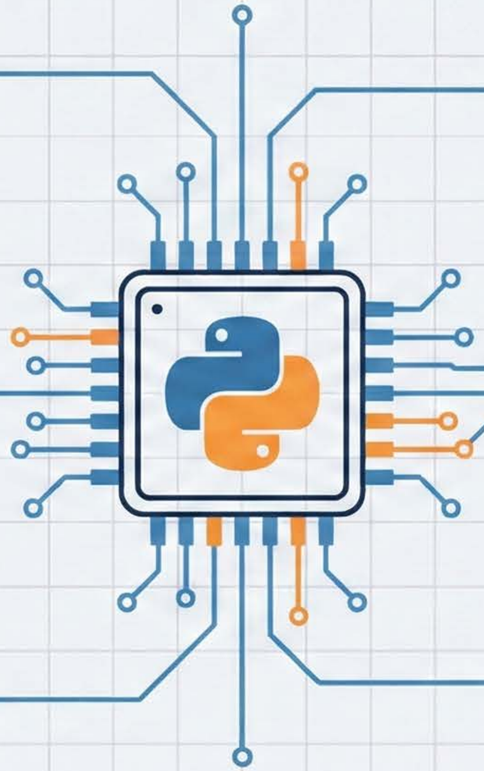
NotebookLM

為什麼 Python 是化工與資料科學的首選？

語法簡潔 (Readability)
接近自然語言，易於維護

社群支持 (Community)
龐大的開發者社群，容易找到解決方案

跨平台 (Cross-Platform)
Windows/macOS/Linux 皆可運行



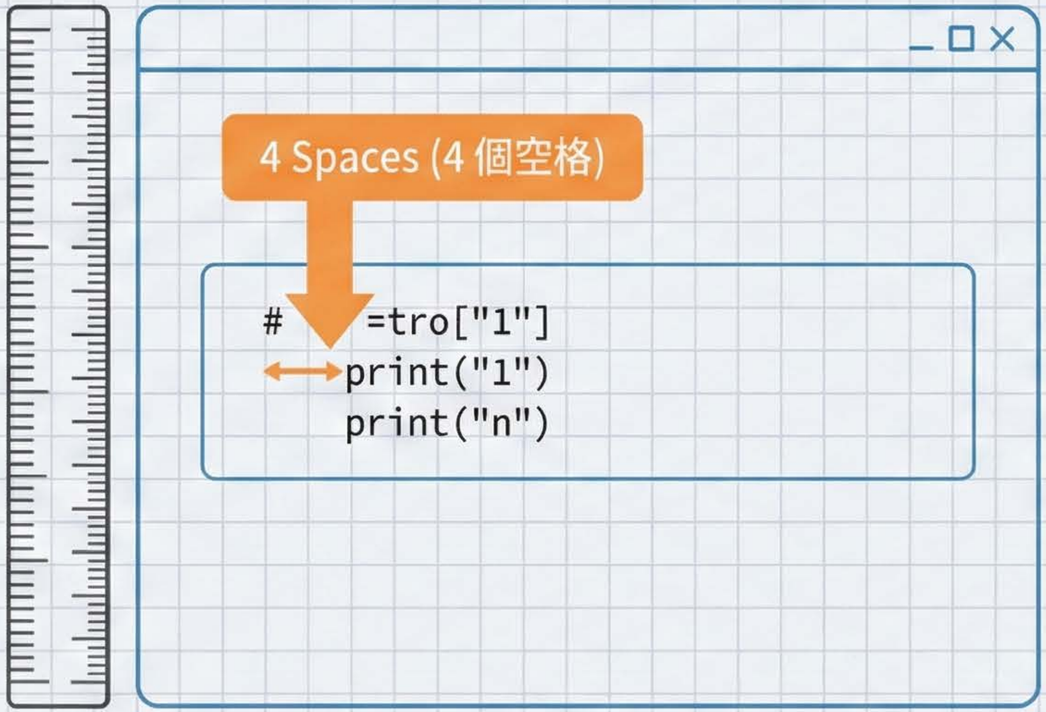
豐富生態 (Ecosystem)
擁有強大科學運算套件

工業標準 (Industry Standard)
科技巨頭與研究機構主流選擇

“Python 不僅僅是程式語言，它是連接化工原理與人工智慧的通用語言。”

奠定基礎：語法結構與數據容器

程式碼結構 (Structure)



縮排 (Indentation)：必須保持一致 (4 個空格)

註解 (Comments)：# 後的文字，機器會忽略

變數與資料型態 (Variables)

數值 (Numeric)

Float: Temperature = 25.4

字串 (String)

Tag_ID = 'P-101'

布林值 (Boolean)

Valve_Open = True

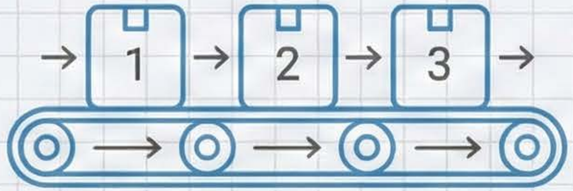
命名規則 (Naming Convention)

使用小寫與底線 (snake_case)，例如 reactor_temp

NotebookLM

資料結構：組織與管理數據的方式

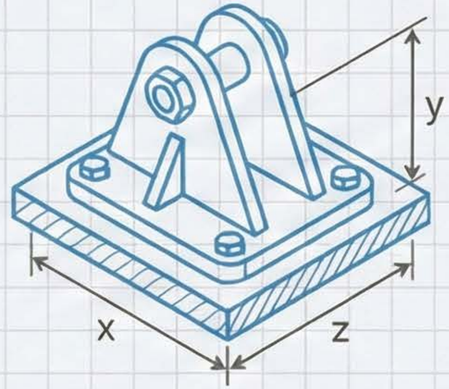
1. 串列 (List)



`[1, 2, 3]`

有序、可變 (Mutable)。類似輸送帶上的連續樣品。

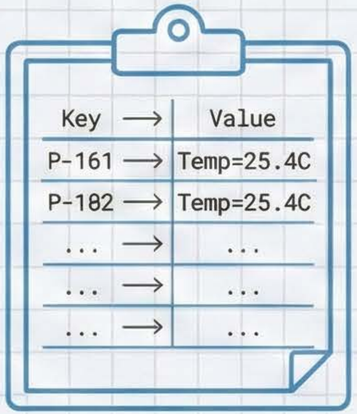
2. 元組 (Tuple)



`(x, y, z)`

有序、不可變 (Immutable)。類似固定的設備座標。

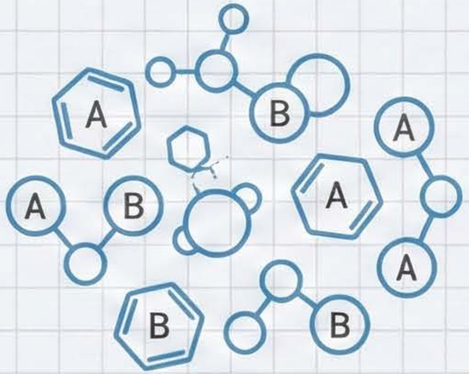
3. 字典 (Dictionary)



`{'key': 'value'}`

鍵值對應。類似物性查表。

4. 集合 (Set)



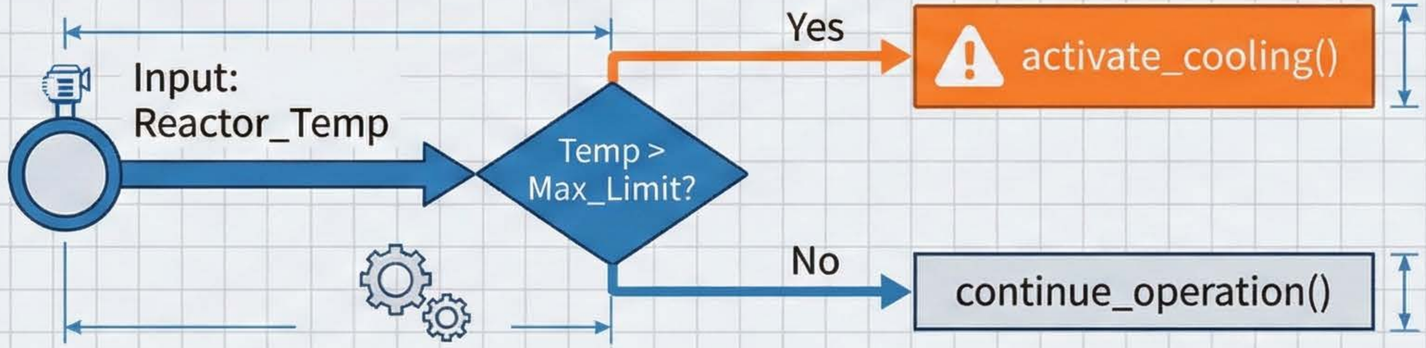
`{A, B}`

無序、不重複。類似獨特化學成分列表。

NotebookLM

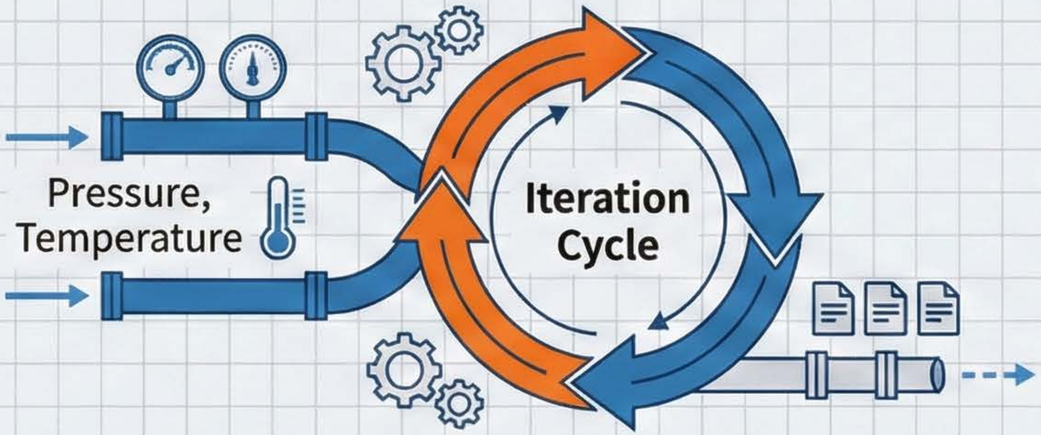
控制流程：自動化邏輯與批次處理

邏輯判斷 (Conditionals) - 數位安全連鎖系統




```python
if temperature > 120:
    status = 'Warning'
else:
    status = 'Normal'
```

迴圈 (Loops) - 自動化批量處理



For 迴圈

已知次數的重複。

```python
for sensor in sensor_list:
    read(sensor)
```

While 迴圈

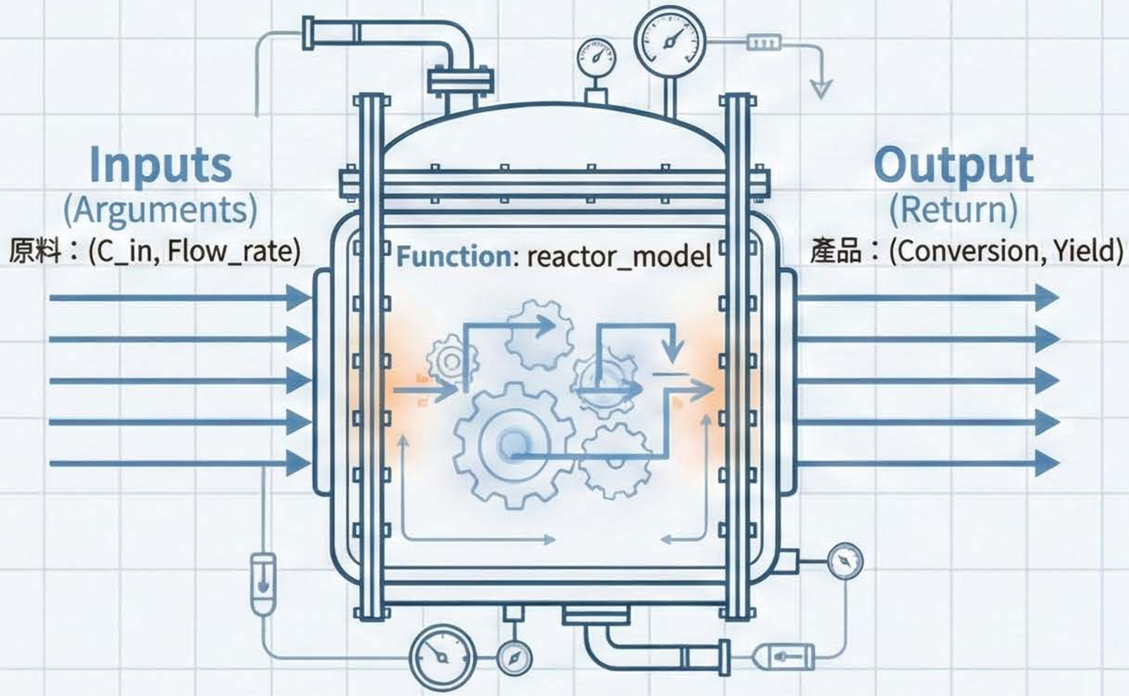
執行直到條件滿足。

```python
while concentration < target:
    add_reagent()
```

NotebookLM

函式與模組：單元操作與巨人的肩膀

函式 (Functions) = 單元操作



- 定義 (Def)：使用 def 宣告，避免重複造輪子 (DRY Principle)。

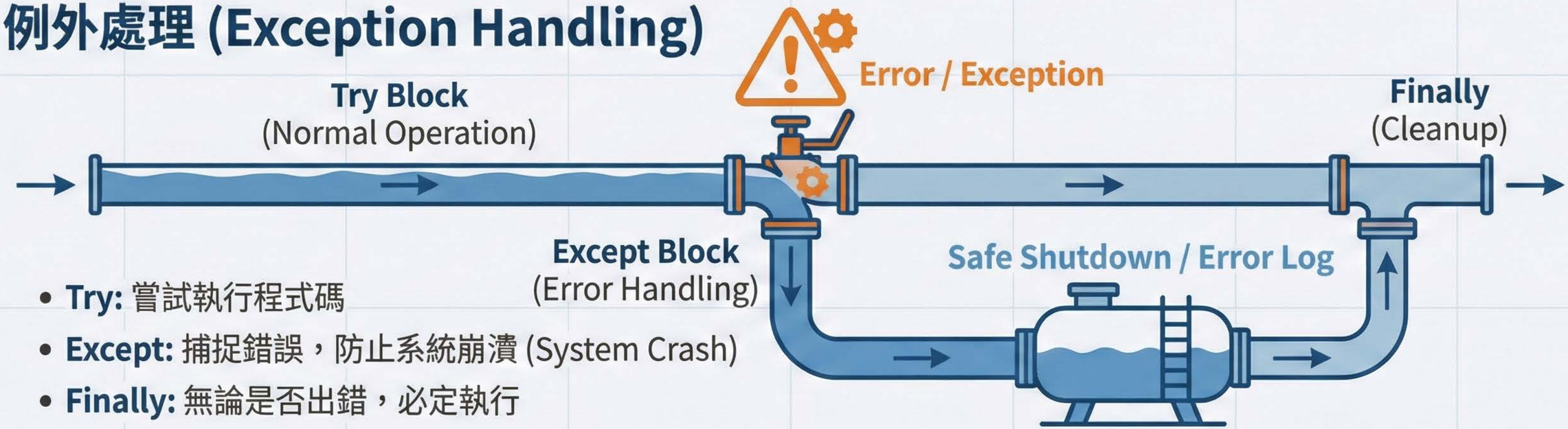
模組 (Modules) = 零件庫存

```
import numpy as np
```

NotebookLM

系統穩健性與最佳實踐

例外處理 (Exception Handling)



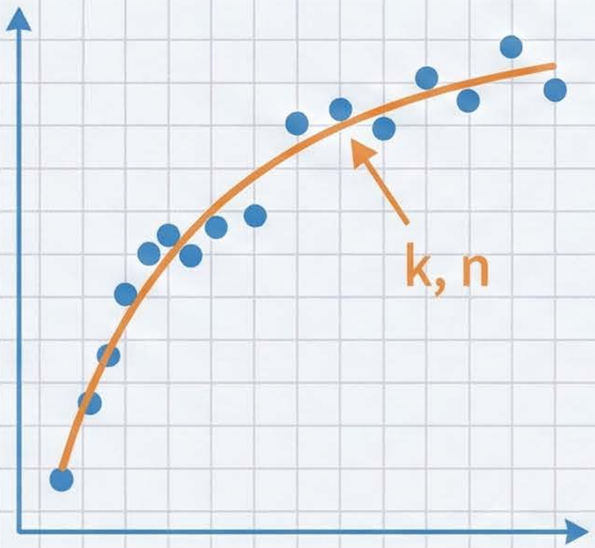
- **Try:** 嘗試執行程式碼
- **Except:** 捕捉錯誤，防止系統崩潰 (System Crash)
- **Finally:** 無論是否出錯，必定執行

最佳實踐 (Best Practices)

- ☑ **PEP 8 風格指南**
- ☑ **有意義的命名** (T_inlet 優於 t)
- ☑ **除錯 (Debugging):** 閱讀

化工領域的 Python 實際應用

反應動力學擬合 (Kinetics)

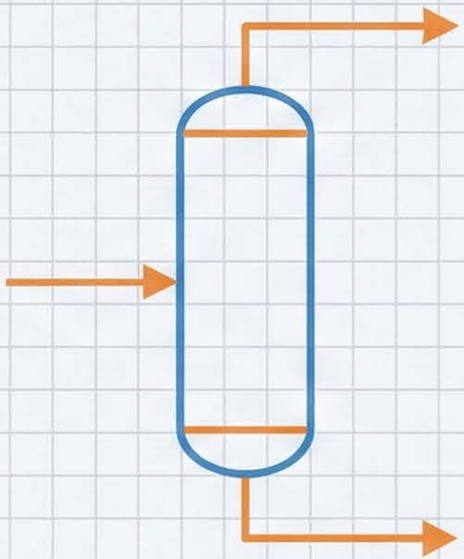


使用 SciPy 擬合實驗數據，求出反應速率常數 k 與級數 n。

異常檢測 (Anomaly Detection)

監控製程數據，自動識別偏離常態的孤立點 (Outliers)。

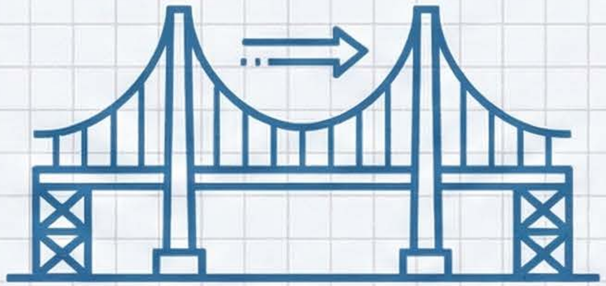
單元操作模擬 (Simulation)



模擬蒸餾塔或 CSTR 在不同條件下的穩態輸出。

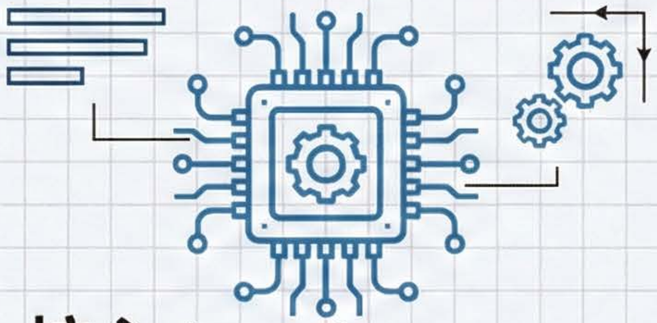
NotebookLM

結語：您的數位工具箱已備妥



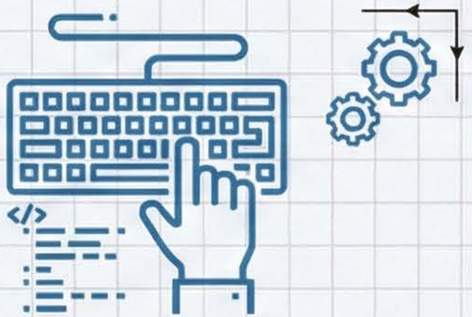
橋樑 (Bridge)

Python 是連結化工原理與數據驅動決策的橋樑。



核心 (Core)

掌握變數、迴圈與函式，即掌握了自動化的核心。



實踐 (Practice)

理論是不夠的，您需要親自寫 Code。

下一步 (Next Step)：
開啟 `Unit02_Python_Basics.ipynb`
現在開始您的實作練習！